

Projeto Integrador V-B

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas

Discente: José Humberto Curtiço Júnior

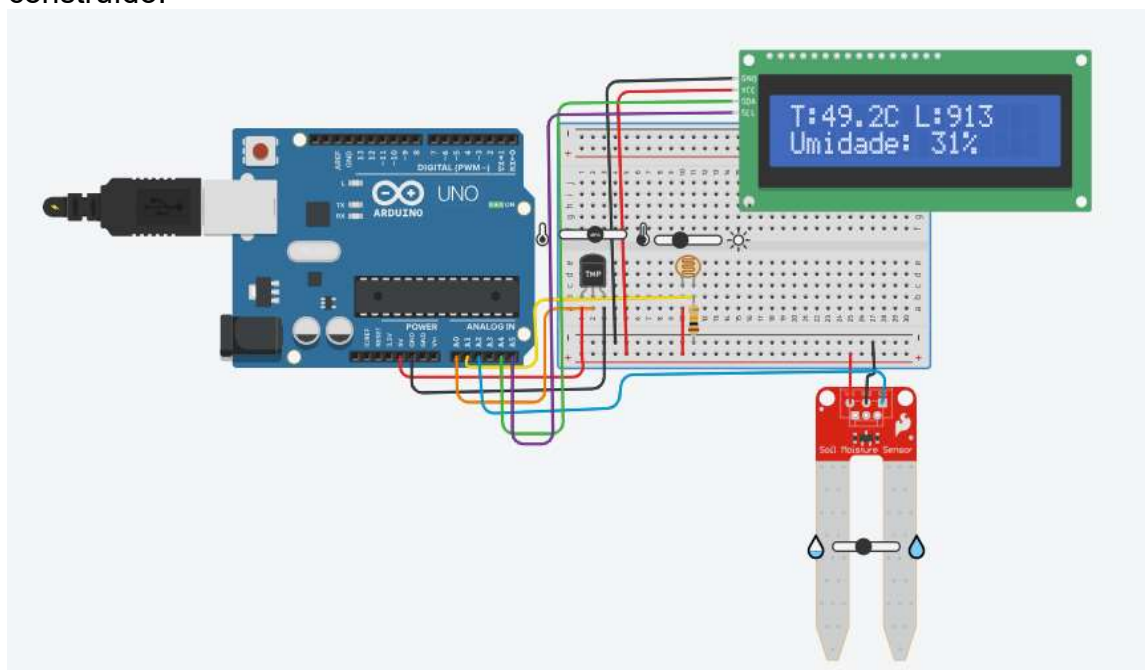
Professor Esp. Thalles Bruno Gonçalves Nery dos Santos

Relatório Técnico: Sistema Integrado de Monitoramento Residencial Inteligente

Este projeto apresenta a concepção e prototipação de um sistema “Smart Home” focado no monitoramento do ambiente de uma residência. O objetivo é coletar, processar e exibir dados em tempo real (temperatura, umidade e luminosidade) através de uma arquitetura que integra hardware (IoT com Arduino), processamento lógico (Java) e uma interface de usuário intuitiva.

Esquema Ilustrativo do Protótipo Arduino e Componentes

O protótipo de hardware foi desenvolvido utilizando o simulador Tinkercad, atuando como a camada de coleta de dados físicos do ambiente. O microcontrolador central, um Arduino UNO R3, executa o firmware, gerencia as leituras analógicas e exibe os dados. Para a captação, utiliza-se um Sensor de Temperatura (TMP36), que emite um sinal de tensão linearmente proporcional aos graus Celsius, e um Fotorresistor (LDR) com resistor de 10k Ω em configuração de divisor de tensão para mensurar a intensidade luminosa. Devido às limitações da biblioteca nativa do Tinkercad para sensores DHT11/22, a umidade do ar foi representada de forma simulada e funcional por um Sensor de Umidade de Solo, permitindo validar a lógica do sistema. Os dados processados (Temperatura, Umidade e Luz) são apresentados localmente em um Display LCD 16x2, garantindo o monitoramento direto sem dependência exclusiva de um terminal serial externo. Abaixo, apresenta-se o diagrama de conexões do circuito construído:



Link para visualizar no Tinkercad:

https://www.tinkercad.com/things/crPgh4AVzox-sensor-de-temperatura-umidade-e-luminosidade?sharecode=tQoINxPRG4wXIKi1QEQoubEr9LhyDCAp7MiXc9_M0EA

Lógica de Programação e Código (Arduino)

O firmware, desenvolvido em C++ com a biblioteca LiquidCrystal.h, opera através da amostragem contínua das portas analógicas A0, A1 e A2. Ocorre então o processamento local, que converte matematicamente os valores brutos (0-1023) para escalas físicas (Celsius e porcentagens). Em seguida, os resultados são impressos no Display LCD para monitoramento local e enviados simultaneamente via comunicação serial para integração com o módulo Java, num ciclo temporizado de 5 segundos que garante estabilidade nas leituras.

Segue abaixo o código:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Inicialização do LCD: endereço 0x27 ou 0x3F, 16 colunas e 2 linhas
LiquidCrystal_I2C lcd(32, 16, 2);
// Definição dos pinos analógicos
const int pinTemp = A0, pinLuz = A1, pinUmidade = A2;

void setup() {
  Serial.begin(9600); // Inicialização da comunicação Serial (Baud rate 9600 para Java)
  lcd.init(); lcd.backlight(); // Inicialização do LCD
  lcd.setCursor(0, 1); lcd.print("Inicializando...");
  delay(2000);
}

void loop() {
  // 1. Processamento da Temperatura (Sensor TMP36)
  int leituraTemp = analogRead(pinTemp);
  float temperaturaC = (((leituraTemp * (5.0 / 1023.0)) - 0.5) * 100.0);

  // 2. Processamento da Iluminação (LDR)
  int leituraLuz = analogRead(pinLuz);

  // 3. Processamento da Umidade (Mapeia o valor analógico de 0-1023 para 0-100%)
  int porcentagemUmidade = map(analogRead(pinUmidade), 0, 1023, 0, 100);

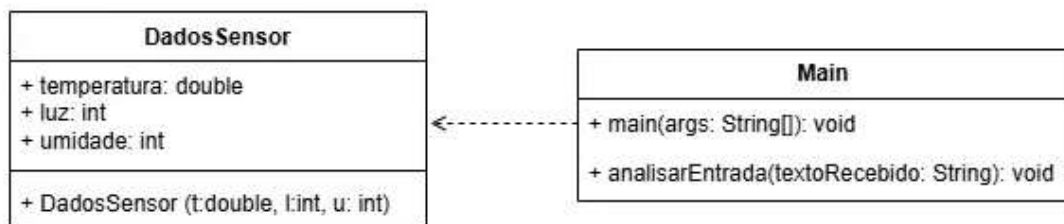
  // --- Atualização do Display LCD ---
  lcd.clear();
  lcd.setCursor(0, 0); // Linha 1: Temperatura e Luz
  lcd.print("T:"); lcd.print(temperaturaC, 1); lcd.print("C L:"); lcd.print(leituraLuz);
  lcd.setCursor(0, 1); // Linha 2: Umidade
  lcd.print("Umidade: "); lcd.print(porcentagemUmidade); lcd.print("%");

  // --- Envio de dados via Serial para o Módulo Java ---
  // Formato: Temperatura;Luz;Umidade
  Serial.print(temperaturaC); Serial.print(";");
  Serial.print(leituraLuz); Serial.print(";"); Serial.println(porcentagemUmidade);

  delay(5000); // Intervalo de 5 segundos entre leituras
}
```

Módulo em Java: Diagrama de Classes e Código Fonte

A arquitetura orientada a objetos organiza o sistema de forma modular, conforme detalhado no diagrama UML. A classe DadosSensor atua como o modelo de dados para o encapsulamento dos atributos ambientais, enquanto a classe Main funciona como o núcleo orquestrador. Por meio de uma relação de dependência, a Main coordena o fluxo de execução e consome as informações instanciadas na DadosSensor. Essa estrutura garante a separação entre a representação do dado e sua manipulação lógica, estabelecendo a base necessária para a futura implementação de classes especialistas e serviços de comunicação.



Link para visualizar o diagrama:

https://drive.google.com/file/d/1WT_LnZ1IRBnQl1czc3Go2lyVOfdwOFq8/view?usp=sharing

Como o hardware não está fisicamente conectado nesta fase, a solução emprega dados simulados (mockados) via classe `java.util.Random`, gerados em faixas realistas e formatados em uma `String` separada por ponto e vírgula (;), mimetizando a porta serial do Arduino. A lógica de controle realiza o *parse* dessa `String`, converte os tipos e instancia o modelo de dados, aplicando em seguida as regras de negócio via condicionais para informar a interpretação sobre as condições do ambiente. Segue abaixo o código:

```
import java.util.Random;

public class DadosSensor {
    public double temperatura;
    public int luz;
    public int umidade;

    public DadosSensor(double t, int l, int u) {
        this.temperatura = t; this.luz = l; this.umidade = u;
    }
}

public class Main {
    public static void main(String[] args) {
        Random sorteador = new Random();
        System.out.println("===== MONITORAMENTO DO AMBIENTE =====");

        // Repetição para simular 10 leituras conforme o tempo passando
        for (int i = 1; i <= 10; i++) {
            // Criando valores aleatórios (falsos sensores)
            double tempFake = 10 + (sorteador.nextDouble() * 30);
            int luzFake = sorteador.nextInt(1024); // Luz entre 0 e 1023
            int umidFake = sorteador.nextInt(101); // Umidade entre 0 e 100

            //String no formato que seria recebido pelo Arduino
```

```

String linhaDoArduino = String.format("%.1f", tempFake).replace(",", ".") + ";" +
luzFake + ";" + umidFake;

System.out.println("Leitura #" + i);
analisarEntrada(linhaDoArduino);

try { Thread.sleep(5000); /* Pausa de 5 segundos entre as leituras */ }
catch (InterruptedException e) { System.out.println("Erro na pausa!"); }
}
}

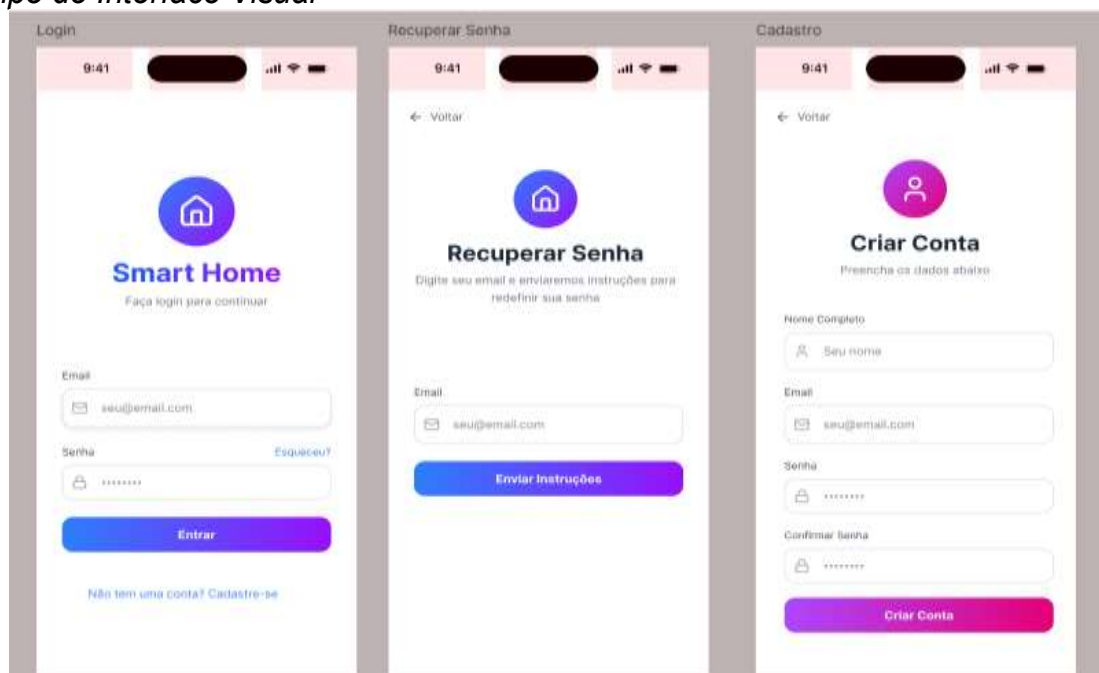
public static void analisarEntrada(String textoRecebido) {
String[] partes = textoRecebido.split(";");
double temp = Double.parseDouble(partes[0]);
int luz = Integer.parseInt(partes[1]);
int umid = Integer.parseInt(partes[2]);

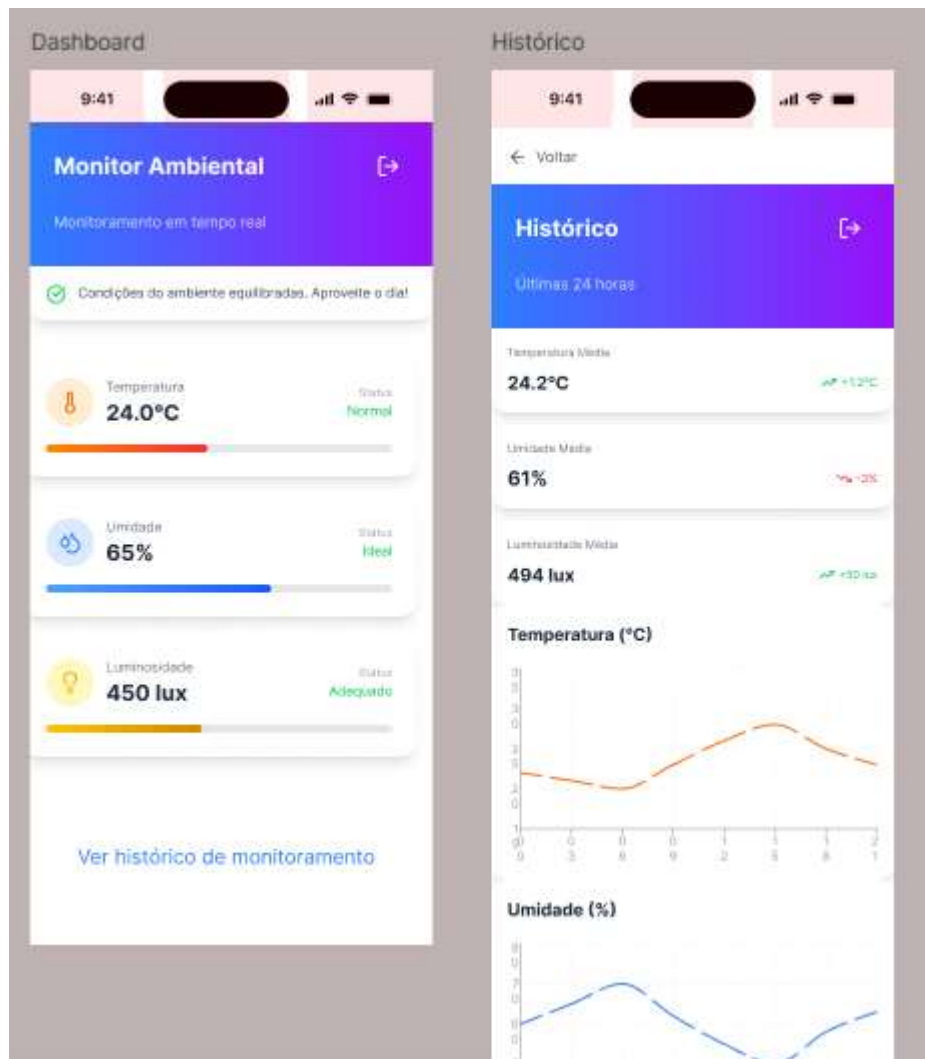
DadosSensor dados = new DadosSensor(temp, luz, umid);
System.out.println("Status: T:" + dados.temperatura + "°C L:" + dados.luz + " U:" +
dados.umidade + "%");

// ANÁLISE DAS MEDIÇÕES
if (dados.temperatura < 18) System.out.println("A temperatura esta baixa. Ligue o
aquecedor ou pegue um agasalho.");
if (dados.temperatura >= 30) System.out.println("Temperatura alta! Ligue o ar
condicionado!");
if (dados.luz < 300) System.out.println("Ambiente escuro. Acenda as luzes!");
if (dados.umidade <= 30) System.out.println("O ar esta muito seco. Ligue o
umidificador e mantenha-se hidratado!");
if (dados.temperatura >= 18 && dados.temperatura < 30 && dados.luz >= 300 &&
dados.umidade > 30) {
System.out.println("Condicoes do ambiente equilibradas. Aproveite o dia!");
}
}
System.out.println("-----");
}
}
}

```

Protótipo de Interface Visual





Link do protótipo no Figma:

<https://www.figma.com/proto/mzaQcAxeEs3HNYGnuwn7dD/Sistema-de-monitoramento-de-ambiente?node-id=0-1&t=4o7zBEB6nzZcvBcL-1>

O protótipo desenvolvido no Figma traduz visualmente toda a lógica do sistema, conectando as telas de acesso — como login, cadastro e recuperação de senha — aos módulos práticos de monitoramento e histórico. A estrutura segue uma linha minimalista para evitar poluição visual e focar no que é essencial para o usuário, facilitando a leitura rápida e intuitiva dos índices de temperatura, luz e umidade em um dashboard limpo.

Um ponto central da interface é que ela não se limita a exibir números, mas mantém um campo de feedback constante baseado na análise processada pelo módulo Java. Essa área de notificações permanece visível na dashboard o tempo todo, transformando os dados brutos dos sensores em recomendações práticas em tempo real. Dessa forma, quando o sistema identifica condições como temperatura baixa ou falta de luminosidade, o usuário recebe imediatamente o alerta com a ação sugerida, integrando o processamento do backend a uma experiência de uso direta e funcional.

Link para vídeo de apresentação no youtube: <https://youtu.be/OO-GjuRSnwa>